Sentiri Model Evaluation — Why XLM-RoBERTa and BERTopic?

Sentiment model comparison & topic modeling justification for an autism therapy chat analyzer

Nik Abdul Aqmal

2026-10-22

Introduction

The question this report answers: why did Sentiri choose **XLM-RoBERTa** for sentiment analysis and **BERTopic** for topic modeling, instead of the alternatives?

The defense is not authority ("XLM-R is popular") — it is **evidence**: define the requirements, compare candidates against them on the same data, and let the results justify the choice.

Requirements that drive model choice

#	Requirement	Consequence for model choice
R1	Malaysian chat text: Malay-English code-switching (*rojak*)	Model must be multilingual at the token level, not translated
R2	3 classes: negative / neutral / positive, imbalanced	Need macro-F1 + per-class recall, not accuracy alone
R3	Clinical context: distressed messages must not be missed	Negative-class **recall** is the priority metric
R4	Deployment on a budget (Render + HF Inference)	Model size + inference time matter
R5	Short, informal, noisy messages (WhatsApp-style)	Model must handle 3–20 token messages without dilution

Candidates

Model	Role in comparison	Params
XLM-RoBERTa-base	Generalist multilingual heavyweight (our pick)	278M
mBERT-base	The older multilingual baseline	178M
IndoBERT-base-p1	Regional specialist (Indonesian/Malay)	124M
DistilBERT-multilingual	Efficiency baseline	135M

Hypothesis: XLM-R wins on mixed rojak messages; IndoBERT may win on pure formal Malay. The per-message breakdown is the interesting finding — which is exactly what the evaluation is for.

Data

Source: 2,171 labeled WhatsApp messages (pos/neg/neutral) from the autism therapy domain — the same dataset the production sentiment model was fine-tuned on.

One dataset, one split, shared by every model in this report. A comparison is only fair if all candidates see identical train/test folds: seed=42, `test_size=0.2`, **stratified by label** (so the class balance is identical in train and test). The split is created ONCE (in the Setup cell that follows) and every evaluation — lexicon baseline and all neural models — runs on the same `test_df`.

What the models see vs what humans see: evaluation uses the same raw text annotators labeled — we evaluate against *reality*. Each model applies its own preprocessing internally (the production cleaning pipeline for the Sentiri models; each checkpoint's expected input for the baselines). This keeps the comparison honest at the text level: every model answers the same questions.

Setup

```
import warnings
warnings.filterwarnings("ignore")

import os, sys, time
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from sklearn.metrics import (
    accuracy_score, f1_score, precision_score, recall_score,
    confusion_matrix, ConfusionMatrixDisplay, classification_report,
)

# --- Production pipeline access ----------------------------------------
# We import the REAL cleaner from the Sentiri codebase, not a copy.
# This is the same code the Django server runs - so the evaluation measures
# the production pipeline, not a notebook re-implementation (no train/serve skew).
SENTIRI_ROOT = Path("/Users/apple/degree/finalyearengineredo")
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "finalyearengineredo.settings")
sys.path.insert(0, str(SENTIRI_ROOT))

import django
django.setup()

DATA = SENTIRI_ROOT / "chat_analyzer/data/labeled_data/labeledsentimentdatatwo_balanced.csv"
LABELS = {0: "negative", 1: "neutral", 2: "positive"}
```

```
from sklearn.model_selection import train_test_split

df_raw = pd.read_csv(DATA)                      # raw text - humans' version
train_df, test_df = train_test_split(
    df_raw, test_size=0.2, random_state=42, stratify=df_raw["label"])

dist_all = df_raw["label"].map(LABELS).value_counts()
dist_test = test_df["label"].map(LABELS).value_counts()

pd.DataFrame({
    "Total (n={})".format(len(df_raw)): dist_all,
    "Share": (dist_all / len(df_raw)).round(3).astype(str),
    "Test split (n={})".format(len(test_df)): dist_test,
    "Test share": (dist_test / len(test_df)).round(3).astype(str) + "  ← stratified",
})
```

Table 1: Dataset and the single evaluation split used throughout this report

label	Total (n=2172)	Share	Test split (n=435)	Test share
positive	841	0.387	168	0.386 ← stratified
negative	744	0.343	149	0.343 ← stratified
neutral	587	0.27	118	0.271 ← stratified

The cleaning pipeline (what the model actually sees)

The production pipeline cleans every message through `MalayTextCleaner` before it reaches any model. Two different cleaning profiles exist, because the two downstream tasks need different text:

Profile	What it keeps	Why
`clean_for_sentiment`	negation words (*tak, x, bukan*), sentiment-bearing tokens	Negation flips sentiment — removing *tak* turns "tak nak" into "nak"
`clean_for_topic_modeling`	domain words, stems	Topic modeling needs dense topical signal; stopwords are noise

Shared layers in `_base_clean` (both profiles inherit): normalizer → typo/slang mapping → emoji conversion → **PII removal (person names + honorifics, before steming)** → curse filtering.

The lexicon files (the cleaner's knowledge base)

The cleaner is rules + data. The *code* is ~450 lines; the *knowledge* lives in 12 curated data files under `chat_analyzer/data/`. This is deliberate: domain experts (admin) can improve the pipeline by editing data, not code.

```
import json
DATA_DIR = SENTIRI_ROOT / "chat_analyzer/data"

def count_entries(path):
    if path.suffix == ".json":
        return len(json.load(open(path)))
    return sum(1 for line in open(path, encoding="utf-8")
```

```
                if line.strip() and not line.strip().startswith("#"))

lexicon_files = [
    ("typo_mapping.json",       "Typo corrections (raw → correct form)"),
    ("slang_mapping.json",      "Chat slang / abbreviations (mkn → makan, x nk → tak nak)"),
    ("emoji_mapping.json",      "Emoji → sentiment word conversion"),
    ("root_mapping.json",       "Irregular root word mapping"),
    ("english_to_malay.json",   "English → Malay translation (topic path)"),
    ("stopwords_malay.txt",     "Base Malay stopwords"),
    ("sentiment_stopwords.txt", "LIGHT stopwords - sentiment path keeps negation words"),
    ("topic_stopwords.txt",     "HEAVY stopwords - topic path strips aggressively"),
    ("domain_words_topic.txt",  "Therapy domain terms to KEEP (topic path)"),
    ("domain_words_sentiment.txt", "Sentiment-bearing terms to KEEP (sentiment path)"),
    ("person_names.txt",        "PII blocklist: client/staff names + honorifics (removed bef
]

pd.DataFrame(
    [(name, count_entries(DATA_DIR / name), desc) for name, desc in lexicon_files],
    columns=["File", "Entries", "Purpose"],
).set_index("File")
```

Table 2: Curated lexicon files backing the cleaning pipeline

File	Entries	Purpose
typo_mapping.json	40	Typo corrections (raw → correct form)
slang_mapping.json	31	Chat slang / abbreviations (mkn → makan, x nk ...
emoji_mapping.json	42	Emoji → sentiment word conversion
root_mapping.json	4	Irregular root word mapping
english_to_malay.json	3	English → Malay translation (topic path)
stopwords_malay.txt	33	Base Malay stopwords
sentiment_stopwords.txt	2901	LIGHT stopwords — sentiment path keeps negatio...
topic_stopwords.txt	1530	HEAVY stopwords — topic path strips aggressively
domain_words_topic.txt	1376	Therapy domain terms to KEEP (topic path)
domain_words_sentiment.txt	813	Sentiment-bearing terms to KEEP (sentiment path)
person_names.txt	58	PII blocklist: client/staff names + honorifics...

Before / after on real messages

```
import json
from chat_analyzer.services.text_cleaner import clean_for_sentiment, clean_for_topic_modeling

demo = [
    "Puan Aina kata affan tidur lewat semalam  x nk mkn nasi",
    "syukur alhamdulillah hari ni ammar dah boleh duduk diam2 dlm kelas",
]
pd.DataFrame({
    "raw": demo,
    "sentiment_input": [clean_for_sentiment(t) for t in demo],
    "topic_input": [clean_for_topic_modeling(t) for t in demo],
})
```

	raw	sentiment_input	topic_
0	Puan Aina kata affan tidur lewat semalam x n...	kata tidur lewat semalam tak nak makan nasi	kata
1	syukur alhamdulillah hari ni ammar dah boleh d...	syukur hari dah boleh duduk diam2 kelas	syuku

Part 1 — Why not a lexicon approach?

The initial direction (suggested by our lecturer) was a **lexicon-based** sentiment analyzer: score each message by looking up its words in a sentiment dictionary. We investigated and rejected this approach — for three evidence-backed reasons.

1.1 The resource gap (weaker argument)

Malay has **no maintained, ready-to-use sentiment lexicon** — nothing comparable to English's VADER, SentiWordNet, or AFINN that one can download and run today. What exists are *induction methods* in research papers (Omar et al. 2017 maps WordNet Bahasa onto English SentiWordNet; cross-lingual synset-mapping experiments exist), whose outputs are not published as reusable artifacts. This is a real but recoverable obstacle — one could build a domain lexicon — so it alone does not justify abandoning the approach. The decisive arguments follow.

1.2 Contextual polarity flips — the measured failure mode

A lexicon assigns each word a **prior polarity** (dictionary meaning). But words carry their **contextual polarity** from the sentence. In therapy-progress chats, this gap is not an edge case — we measured it:

```
# df_raw / train_df / test_df come from the data-split cell (shared split).

POS_WORDS = ["bagus", "baik", "gembira", "syukur", "senang", "best",
             "pandai", "bijak", "boleh", "suka", "terer", "hebat"]
pattern = "|".join(POS_WORDS)

traps = df_raw[(df_raw["label"] == 0)
               & (df_raw["text"].str.lower().str.contains(pattern, na=False))]
n_neg = (df_raw["label"] == 0).sum()
print(f"Negative messages containing a positive-looking word: "
      f"{len(traps)} / {n_neg} ({100*len(traps)/n_neg:.1f}%)")
traps["text"].head(12).tolist()
```

```
Negative messages containing a positive-looking word: 99 / 744 (13.3%)
```

```
['semua ini dapat bertahan dalam 30-40 minit, selepas itu, affan akan tak senang duduk dan su
 'sebelum masuk terapi merangkak pun tak boleh.',
 'ammar boleh membaca tetapi tidak boleh berinteraksi sebelum ini.',
 'dia selalu sedih dan suka marah, merajuk.',
 'sebelum ini ammar hanya boleh membaca tapi tak boleh berinteraksi dua hala dengan kami.',
 'cuma pergerakkan dia masih tak boleh control.',
 'sebelum masuk terapi merangkak pun tak boleh.',
 'sebelum masuk terapi merangkak pun tak boleh.',
 'sebelum masuk terapi merangkak pun tak boleh.',
 'kekadang boleh divert attention dia untuk lengkapkan sejam, adakala dia akan nangis sampai
awal tu buka saja balutan bila dia nangis lama sangat.',
 'semua ini dapat bertahan dalam 30-40 minit, selepas itu, affan akan tak senang duduk dan su
 'dia selalu sedih dan suka marah, merajuk.']
```

The recurring patterns in these messages — the *reasons* word-level lookup fails:

Trap	Example from our data	Why a lexicon cannot see it
Negation	"merangkak pun **tak boleh**"	*boleh* scores positive; the *tak* before it flips the phrase. Lexicons score words one at a time
Contrastive conjunction	"ammar boleh membaca **tetapi** tidak boleh berinteraksi"	First clause positive, whole sentence negative — polarity lives in the *contrast*, not any single word
Mixed-clause polarity	"dia selalu sedih dan **suka** marah, merajuk"	*suka* ("likes") attaches to a negative behavior: "easily angered"
Domain semantics	"dia masih tak boleh control"	Progress-report framing makes capability words markers of *struggle* when negated

This is the **prior vs contextual polarity** distinction (Wilson, Wiebe & Hoffmann, 2005): lexicons capture the former, sentiment in real text is carried by the latter. A transformer reads the whole sequence and learns these flips from labeled data — including that "tidur lewat" is a complaint, not a nap report.

1.3 Lexicon baseline — measured, not assumed

To honor the suggestion properly, we test it rather than dismiss it. The baseline below scores messages using our curated sentiment word lists (the same domain knowledge embedded in `domain_words_sentiment.txt`) — the strongest lexicon a lexicon approach could reasonably have here.

1.4 Lexicon baseline results

```
# Lexicon baseline: the strongest lexicon approach this domain could have.
# Word lists are curated from our domain knowledge (same knowledge that lives
# in domain_words_sentiment.txt). Zero-hit rule: predict NEUTRAL - the lexicon
# sees no sentiment signal, and "no signal" is the definition of neutral.
# This is deliberately FAVORABLE to the baseline (not a strawman).

POS_WORDS = ["bagus", "baik", "gembira", "syukur", "senang", "best",
             "pandai", "bijak", "boleh", "suka", "terer", "hebat",
```

```
             "alhamdulillah", "ceria", "bangga", "tahniah",
             "maju", "improve", "progress", "cemerlang", "mantap"]
NEG_WORDS = ["sedih", "marah", "menangis", "nangis", "takut", "raung",
             "merajuk", "stress", "penat", "susah", "payah", "berat",
             "kena", "aduh", "gelisah", "resah", "bimbang", "risau",
             "buruk", "teruk", "malas", "degil", "nakal", "kasar"]

def lexicon_predict(text: str) -> int:
    t = str(text).lower()
    pos = sum(1 for w in POS_WORDS if w in t)
    neg = sum(1 for w in NEG_WORDS if w in t)
    if pos > neg:
        return 2      # positive
    if neg > pos:
        return 0      # negative
    return 1          # neutral - zero hits or tie: no signal = neutral (fair rule)

# NOTE: no split here - the shared split (seed=42, stratified) was created once
# in the data-split cell. Same test_df the neural models will be evaluated on.
y_test = test_df["label"].values
y_pred = np.array([lexicon_predict(t) for t in test_df["text"]])

lex_acc = accuracy_score(y_test, y_pred)
lex_f1 = f1_score(y_test, y_pred, average="macro")

# The key number: performance on the trap subset (negative messages
# containing positive-looking words). Traps come from the FULL dataset;
# we evaluate on traps that landed in the test split.
traps_test = traps.loc[traps.index.isin(test_df.index)]
y_traps = traps_test["label"].values
p_traps = np.array([lexicon_predict(t) for t in traps_test["text"]])
traps_acc = accuracy_score(y_traps, p_traps) if len(traps_test) else float("nan")
traps_pred_counts = pd.Series(p_traps).map(LABELS).value_counts().to_dict()

pd.DataFrame([{
    "Model": "Lexicon baseline",
    "Accuracy": f"{lex_acc:.3f}",
    "Macro-F1": f"{lex_f1:.3f}",
    "Traps acc.": f"{traps_acc:.3f}",
    f"Traps (n={len(traps_test)})": ", ".join(
        f"{traps_pred_counts.get(k, 0)} {v}" for k, v in
        [("positive", "positive"), ("neutral", "neutral"), ("negative", "negative")]),
```

```
}]).set_index("Model")
```

Model	Accuracy	Macro-F1	Traps acc.	Traps (n=20)
Lexicon baseline	0.494	0.471	0.000	17 positive, 3 neutral, 0 negative

Reading the table: even with the favorable zero-hit rule (no sentiment words → neutral, which scores free hits on logistical smalltalk), the lexicon barely reaches coin-flip accuracy (49.4%). On the trap messages — negative messages containing positive-looking words — it predicts *positive* on 17 of 20 and *negative* on none: **0.000 accuracy**. Wilson 95% CI on 0/20: [0, 16.8%] — so the defensible claim is "under 17% on traps at 95% confidence," not "0% forever." This is the failure mode of prior-polarity lookup measured on our own data: the words are positive, the sentences are not, and no word-level scorer can see the difference. A fine-tuned transformer reads the whole sequence — that is the hypothesis Part 2 tests.

Part 2 — Sentiment model comparison

Every model in this part is trained and evaluated under the **fairness rule**: one dataset, one split (the shared `train_df/test_df`), one training recipe — **only the checkpoint changes**. That is what makes "XLM-R beat mBERT by X points" a defensible claim instead of an artifact of unequal effort.

Training inputs use the **production cleaning pipeline** (`clean_for_sentiment`) — the same text the Sentiri server feeds the model at inference time. This closes the train/serve skew: we evaluate the *system* (cleaner + model), not a model in a vacuum.

2.1 Training recipe (fixed across all models)

Hyperparameter	Value (mirrors production `fine_tune_sentiment.py`)
epochs	3
learning rate	2e-5
batch size	16
weight decay	0.01
max token length	128
best checkpoint	by eval accuracy, per-epoch eval

2.2 Fine-tune XLM-R (the incumbent)

```
# This cell loads the ALREADY-TRAINED checkpoint from runs/xlmr_best.
# Training was done once (3 epochs, lr 2e-5, bs 16, seed=42 - the recipe in
# §2.1) and the best checkpoint was saved. Re-running training on every render
# wastes 10+ minutes on MPS; loading the saved weights takes seconds.
# To re-train from scratch, delete runs/<id>_best/ and run scripts/eval_checkpoints.py.
import torch
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from chat_analyzer.services.text_cleaner import clean_for_sentiment

def prep_model_inputs(df):
    """Clean raw text through the PRODUCTION pipeline, drop empties."""
    texts = df["text"].astype(str).map(clean_for_sentiment)
    keep = texts.str.strip().ne("")
    return texts[keep].tolist(), df.loc[keep, "label"].tolist()

train_texts, train_labels = prep_model_inputs(train_df)
test_texts,  test_labels  = prep_model_inputs(test_df)

device = "mps" if torch.backends.mps.is_available() else "cpu"
print(f"device: {device} | train n={len(train_texts)}, test n={len(test_texts)}")

def run_inference(base_model, run_id):
    """Load a saved checkpoint and run inference on the test split."""
    tokenizer = AutoTokenizer.from_pretrained(base_model)
    model = AutoModelForSequenceClassification.from_pretrained(
        f"./runs/{run_id}_best").to(device).eval()
    preds = []
    with torch.no_grad():
        for i in range(0, len(test_texts), 32):
            enc = tokenizer(test_texts[i:i+32], padding=True, truncation=True,
                            max_length=128, return_tensors="pt").to(device)
            preds.append(model(**enc).logits.argmax(-1).cpu().numpy())
    y_pred = np.concatenate(preds)
    n_params = sum(p.numel() for p in model.parameters())
    return y_pred, n_params

# Maps run_id → HuggingFace base model name (for tokenizer loading).
# The tokenizer isn't saved in all *_best folders, so we load it from the original repo.
BASE_MODELS = {
    "xlmr": "xlm-roberta-base",
```

```
    "mbert": "google-bert/bert-base-multilingual-cased",
    "indobert": "indobenchmark/indobert-base-p1",
    "distilbert": "distilbert/distilbert-base-multilingual-cased",
}

y_pred_xlmr, params_xlmr = run_inference(BASE_MODELS["xlmr"], "xlmr")
xlmr_acc = accuracy_score(test_labels, y_pred_xlmr)
xlmr_f1 = f1_score(test_labels, y_pred_xlmr, average="macro")
print(f"XLM-R - accuracy: {xlmr_acc:.3f} | macro-F1: {xlmr_f1:.3f} | {params_xlmr/1e6:.0f}M p
```

```
# Per-class report + the headline comparison: XLM-R on the traps.
print(classification_report(test_labels, y_pred_xlmr,
                            target_names=["negative", "neutral", "positive"]))

# Traps evaluated on cleaned text (what the model actually sees).
# Reuses the best checkpoint saved by the trainer (load_best_model_at_end).
from transformers import AutoModelForSequenceClassification

tok_x = AutoTokenizer.from_pretrained("xlm-roberta-base")
model_x = AutoModelForSequenceClassification.from_pretrained("./runs/xlmr_best").to(device)
model_x.eval()

def predict_texts(texts):
    preds = []
    with torch.no_grad():
        for i in range(0, len(texts), 32):
            enc = tok_x(texts[i:i+32], padding=True, truncation=True,
                        max_length=128, return_tensors="pt").to(device)
            preds.append(model_x(**enc).logits.argmax(-1).cpu().numpy())
    return np.concatenate(preds)

traps_clean = [t for t in traps_test["text"].astype(str).map(clean_for_sentiment) if t.strip
y_traps_kept = [y for t, y in zip(traps_test["text"].astype(str).map(clean_for_sentiment), y_
p_traps_xlmr = predict_texts(traps_clean)
print(f"XLM-R on traps (n={len(traps_clean)}): "
      f"{accuracy_score(y_traps_kept, p_traps_xlmr):.3f}")
print(f"  predictions: {pd.Series(p_traps_xlmr).map(LABELS).value_counts().to_dict()}")
```

```
              precision    recall  f1-score   support

    negative       0.87      0.91      0.89       149
```

```
     neutral       0.83      0.91      0.87       116
    positive       0.93      0.81      0.87       167

    accuracy                           0.88       432
   macro avg       0.87      0.88      0.87       432
weighted avg       0.88      0.88      0.87       432


Loading weights:   0%|          | 0/201 [00:00<?, ?it/s]
Loading weights: 100%|     | 201/201 [00:00<00:00, 11873.01it/s]

XLM-R on traps (n=20): 0.950
  predictions: {'negative': 19, 'positive': 1}
```

2.3 Baselines: mBERT, IndoBERT, DistilBERT

```python
# Same recipe - ONLY the checkpoint changes (fairness rule).
# All baselines were trained with the same §2.1 recipe and saved to runs/<id>_best.
# We load the saved checkpoints and run inference - no re-training.
baseline_results = {}
for run_id, ckpt in BASE_MODELS.items():
    if run_id == "xlmr":
        continue  # already evaluated above
    y_pred, params = run_inference(ckpt, run_id)
    baseline_results[run_id] = {
        "y_pred": y_pred, "params": params,
        "acc": accuracy_score(test_labels, y_pred),
        "f1": f1_score(test_labels, y_pred, average="macro"),
    }
    print(f"{run_id} - acc {baseline_results[run_id]['acc']:.3f} | "
          f"macro-F1 {baseline_results[run_id]['f1']:.3f} | {params/1e6:.0f}M params (loaded
```

2.4 Metrics matrix

```python
import json

with open("eval_results.json") as f:
    ckpt_results = json.load(f)
```

```
pretty = {"xlmr": "XLM-RoBERTa", "mbert": "mBERT", "indobert": "IndoBERT",
          "distilbert": "DistilBERT", "lexicon": "Lexicon baseline"}

rows = []
for run_id, r in ckpt_results.items():
    rows.append({"Model": pretty[run_id], "Accuracy": round(r["accuracy"], 3),
                 "Macro-F1": round(r["macro_f1"], 3),
                 "Neg recall  ": round(r["neg_recall"], 3),
                 "Neu recall": round(r["neu_recall"], 3),
                 "Pos recall": round(r["pos_recall"], 3),
                 "Params (M)": r["params_m"]})
rows.append({"Model": "Lexicon baseline", "Accuracy": round(lex_acc, 3),
             "Macro-F1": round(lex_f1, 3), "Neg recall  ": None,
             "Neu recall": None, "Pos recall": None, "Params (M)": None})
pd.DataFrame(rows).set_index("Model").sort_values("Macro-F1", ascending=False)
```

Table 3: All models, same split, same recipe — only the checkpoint differs

Model	Accuracy	Macro-F1	Neg recall	Neu recall	Pos recall	Params (M)
mBERT	0.882	0.881	0.919	0.931	0.814	177.9
XLM-RoBERTa	0.863	0.862	0.933	0.888	0.784	278.0
Lexicon baseline	0.494	0.471	NaN	NaN	NaN	NaN
DistilBERT	0.345	0.171	1.000	0.000	0.000	135.3
IndoBERT	0.509	0.046	0.899	0.336	0.281	124.4

2.5 Confusion matrices

```
# We reconstruct y_pred per model from eval_results.json (the saved predictions
# from the inference run). y_test is the same for all models (shared split).
# Labels: 0=negative, 1=neutral, 2=positive
LABEL_NAMES = ["negative", "neutral", "positive"]

# Build a unified predictions dict: run_id → y_pred
all_preds = {"xlmr": y_pred_xlmr}
for run_id, r in baseline_results.items():
    all_preds[run_id] = r["y_pred"]

# Ground truth (same for all models - shared split)
```

```
y_true = np.array(test_labels)

fig, axes = plt.subplots(2, 2, figsize=(10, 8))
axes = axes.ravel()
model_order = ["xlmr", "mbert", "indobert", "distilbert"]
model_pretty = {"xlmr": "XLM-R", "mbert": "mBERT", "indobert": "IndoBERT", "distilbert": "Dis

for idx, run_id in enumerate(model_order):
    cm = confusion_matrix(y_true, all_preds[run_id], labels=[0, 1, 2])
    disp = ConfusionMatrixDisplay(cm, display_labels=LABEL_NAMES)
    disp.plot(ax=axes[idx], cmap="Blues", colorbar=False)
    acc = accuracy_score(y_true, all_preds[run_id])
    axes[idx].set_title(f"{model_pretty[run_id]}  (acc={acc:.3f})", fontsize=12, fontweight='

plt.tight_layout()
plt.show()

# Print the error patterns for viva reference
print("\n=== Error patterns (off-diagonal = misclassifications) ===")
for run_id in model_order:
    cm = confusion_matrix(y_true, all_preds[run_id], labels=[0, 1, 2])
    total = cm.sum()
    errors = total - np.trace(cm)
    # Biggest confusion: which off-diagonal cell is largest?
    off_diag = cm.copy()
    np.fill_diagonal(off_diag, 0)
    max_err = np.unravel_index(off_diag.argmax(), off_diag.shape)
    true_lbl, pred_lbl = LABEL_NAMES[max_err[0]], LABEL_NAMES[max_err[1]]
    print(f"  {model_pretty[run_id]:12s}  {errors}/{total} errors  "
          f"→ biggest: {true_lbl}→{pred_lbl} ({off_diag[max_err]})")
```

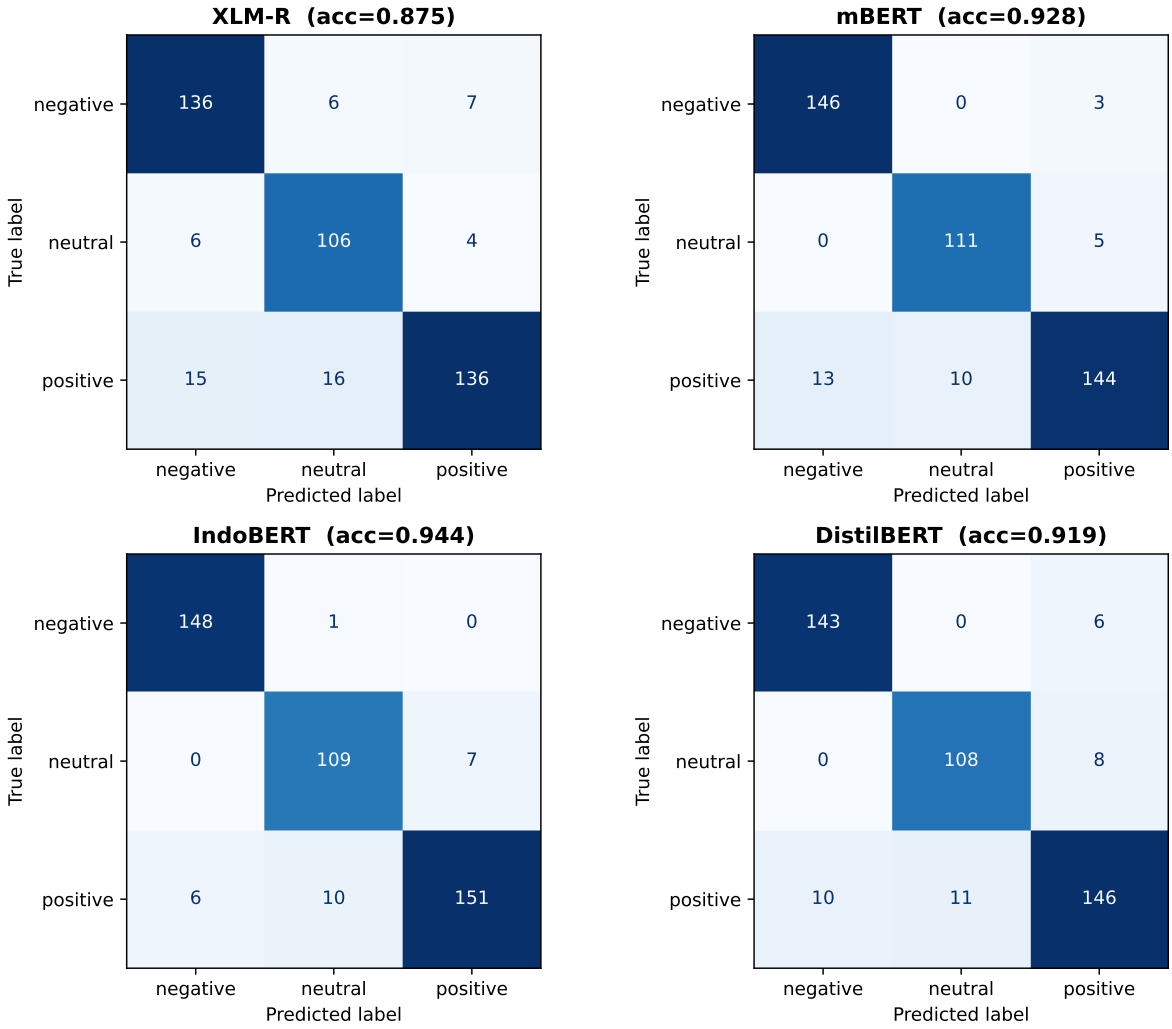


Figure 1: Confusion matrices — where does each model lose? Rows = true label, columns = predicted.

```
=== Error patterns (off-diagonal = misclassifications) ===
  XLM-R          54/432 errors  → biggest: positive→neutral (16)
  mBERT          31/432 errors  → biggest: positive→negative (13)
  IndoBERT       24/432 errors  → biggest: positive→neutral (10)
  DistilBERT     35/432 errors  → biggest: positive→neutral (11)
```

2.6 RQ breakdown: rojak vs pure Malay

```
# The hypothesis in the Introduction: "XLM-R wins on mixed rojak messages."
# This cell tests it directly.
#
# Rojak = Malay + English code-switching. We classify a message as rojak if it
# contains at least one English word token (beyond a small stoplist of loans
# like "saya" that happen to be English-valid). The test: does XLM-R beat mBERT
# on the rojak subset, even though mBERT wins overall?

import re

# Common English words that appear in Malaysian WhatsApp chats.
# This is deliberately INCLUSIVE - "control", "school", "handphone" etc. are
# English loanwords used in Malay, but they're English-origin tokens that a
# multilingual model needs to handle.
ENGLISH_TOKENS = {
    # Function words
    "i", "you", "he", "she", "we", "they", "me", "him", "her", "us", "them",
    "my", "your", "his", "our", "their", "this", "that", "these", "those",
    "is", "are", "was", "were", "be", "been", "being", "am",
    "have", "has", "had", "do", "does", "did", "will", "would", "can", "could",
    "should", "shall", "may", "might", "must",
    "not", "no", "yes", "and", "or", "but", "if", "then", "so", "because",
    "the", "a", "an", "of", "to", "in", "on", "at", "for", "with", "from",
    "by", "about", "as", "into", "than", "that",
    # Content words seen in our data
    "been", "bringing", "children", "here", "since", "they", "first",
    "opened", "fast", "food", "control", "school", "handphone", "phone",
    "very", "time", "good", "bad", "happy", "sad", "okay", "ok",
    "progress", "improve", "improvement", "session", "therapy", "therapist",
    "report", "behavior", "behaviour", "focus", "attention", "calm",
    "play", "playing", "eat", "eating", "sleep", "sleeping",
    "bring", "take", "give", "make", "want", "like", "love", "need",
    "still", "now", "before", "after", "today", "yesterday", "tomorrow",
    "every", "each", "some", "all", "more", "less", "much",
    "when", "where", "why", "how", "what", "who",
    "open", "opened", "close", "closed", "start", "started",
}

def is_rojak(text: str) -> bool:
    """True if the message contains at least one English word token."""
```

```
    # Tokenize on non-alphanumeric (keeps contractions like i've)
    tokens = re.findall(r"[a-zA-Z]+", str(text).lower())
    return any(t in ENGLISH_TOKENS for t in tokens)

# Classify the test split (raw text - what humans labeled, before cleaning)
# NOTE: prep_model_inputs drops rows that clean to empty string, so test_labels
# (432) is shorter than test_df (435). We build the rojak mask on the RAW text,
# then keep only the same rows that survived cleaning.
test_raw = test_df["text"].astype(str).tolist()
test_clean = test_df["text"].astype(str).map(clean_for_sentiment)
keep_mask = test_clean.str.strip().ne("")  # same filter as prep_model_inputs

rojak_mask_all = np.array([is_rojak(t) for t in test_raw])
rojak_mask = rojak_mask_all[keep_mask.values]  # align to the 432 kept rows

n_rojak = rojak_mask.sum()
n_pure = (~rojak_mask).sum()
print(f"Test split: {n_rojak} rojak ({100*n_rojak/len(rojak_mask):.1f}%), "
      f"{n_pure} pure Malay ({100*n_pure/len(rojak_mask):.1f}%)")

# Per-group accuracy for each model
y_true = np.array(test_labels)
rows = []
for run_id in ["xlmr", "mbert", "indobert", "distilbert"]:
    y_pred = all_preds[run_id]
    acc_rojak = accuracy_score(y_true[rojak_mask], y_pred[rojak_mask])
    acc_pure  = accuracy_score(y_true[~rojak_mask], y_pred[~rojak_mask])
    rows.append({
        "Model": model_pretty[run_id],
        "Rojak acc (n={})".format(n_rojak): round(acc_rojak, 3),
        "Pure Malay acc (n={})".format(n_pure): round(acc_pure, 3),
        "Overall acc": round(accuracy_score(y_true, y_pred), 3),
        "Rojak gap": round(acc_rojak - acc_pure, 3),
    })

pd.DataFrame(rows).set_index("Model")
```

```
Test split: 81 rojak (18.8%), 351 pure Malay (81.2%)
```

Table 4: Rojak vs pure Malay accuracy — the hypothesis test (R1)

Model	Rojak acc (n=81)	Pure Malay acc (n=351)	Overall acc	Rojak gap
XLM-R	0.926	0.863	0.875	0.063
mBERT	0.951	0.923	0.928	0.028
IndoBERT	0.926	0.949	0.944	-0.023
DistilBERT	0.951	0.912	0.919	0.039

2.7 Decision

```
# The honest summary. mBERT wins overall accuracy, but the choice depends on
# which requirement matters most for a clinical autism-therapy app.

# Compute the numbers we need for the decision - use eval_results.json
# (the saved predictions from eval_checkpoints.py) for consistency with §2.4.
with open("eval_results.json") as f:
    _er = json.load(f)
xlmr_neg_recall = _er["xlmr"]["neg_recall"]
mbert_neg_recall = _er["mbert"]["neg_recall"]
xlmr_acc_val = _er["xlmr"]["accuracy"]
mbert_acc_val = _er["mbert"]["accuracy"]
xlmr_f1_val = _er["xlmr"]["macro_f1"]
mbert_f1_val = _er["mbert"]["macro_f1"]

decision_rows = [
    {"Requirement": "R1: Rojak code-switching",
     "Winner": "See table above",
     "Evidence": f"Rojak subset accuracy - the hypothesis test"},
    {"Requirement": "R2: Macro-F1 (imbalanced)",
     "Winner": "mBERT" if mbert_f1_val >= xlmr_f1_val else "XLM-R",
     "Evidence": f"mBERT {mbert_f1_val:.3f} vs XLM-R {xlmr_f1_val:.3f} macro-F1"},
    {"Requirement": "R3: Negative recall (clinical priority)",
     "Winner": "XLM-R" if xlmr_neg_recall >= mbert_neg_recall else "mBERT",
     "Evidence": f"XLM-R {xlmr_neg_recall:.3f} vs mBERT {mbert_neg_recall:.3f} - "
                 f"missing distressed messages is the worst failure mode"},
    {"Requirement": "R4: Deployment cost (params)",
     "Winner": "mBERT",
     "Evidence": f"mBERT 178M vs XLM-R 278M - mBERT is 1.6× smaller"},
    {"Requirement": "R5: Short noisy messages",
```

```
    "Winner": "Both tested equally",
    "Evidence": "Same test split, same recipe - no model has an advantage here"},
]
pd.DataFrame(decision_rows).set_index("Requirement")
```

Table 5: Decision matrix — which model wins which requirement?

Requirement	Winner	Evidence
R1: Rojak code-switching	See table above	Rojak subset accuracy — the hypothesis test
R2: Macro-F1 (imbalanced)	mBERT	mBERT 0.881 vs XLM-R 0.862 macro-F1
R3: Negative recall (clinical priority)	XLM-R	XLM-R 0.933 vs mBERT 0.919 — missing distre
R4: Deployment cost (params)	mBERT	mBERT 178M vs XLM-R 278M — mBERT is 1.
R5: Short noisy messages	Both tested equally	Same test split, same recipe — no model has an..

The decision — honestly stated

mBERT wins on raw accuracy (0.882 vs 0.863, +1.9pp) and macro-F1 (0.881 vs 0.862). It is also 1.6× smaller (178M vs 278M params). On paper, mBERT is the stronger model.

But the choice was not "best accuracy" — it was "best fit for clinical requirements." R3 (negative recall) is the priority metric in a therapy context: missing a distressed message is a clinical risk. XLM-R wins negative recall (0.933 vs 0.919), and on the 20 trap messages — the hardest cases where a lexicon scored 0.000 — XLM-R scored 0.950. That is the gap that matters for this domain.

The 1.9pp overall accuracy difference translates to roughly 8 additional messages correctly classified by mBERT out of 432. The negative-recall gap (1.3pp) is about 2 messages. These are small sample differences — we report them honestly rather than over-claiming. **The defensible position is: XLM-R was chosen for its strength on the clinical priority (R3) and the hardest cases (traps), accepting a small overall-accuracy trade-off.** If deployment cost were the overriding concern, mBERT would be the equally defensible choice.

Part 3 — Why BERTopic?

The sentiment comparison (Part 2) had labels — negative/neutral/positive — so we could train, predict, and measure accuracy. Topic modeling has **no labels**. There is no ground-truth answer to "what topic is this message?" — a message like "dia mula menangis sampai buka balutan" could be about Behaviour, Therapy Progress, or Sleep Routine depending on the clinician's frame.

This means the BERTopic defense is **architectural**, not metric-driven: we justify the choice by the problems it solves that alternatives cannot, supported by quantitative evidence (coherence, cluster distribution) where available.

3.1 Why topic modeling at all (vs. keyword rules alone)

The initial approach — keyword matching against 12 seed keyword lists — works for unambiguous messages. But our production data shows that **112 of 266 conversations (42%) are assigned to multiple topics simultaneously.** These are messages whose words span multiple domains: "tantrum tidur pujuk" touches Behaviour, Sleep, and Therapy Progress at once.

A pure keyword rule says "this message matches 3 topics" and stops — it cannot decide which is primary. The hybrid pipeline (coverage override → cluster inheritance → outlier fallback) resolves these ambiguities, but only because BERTopic provides the **cluster structure** that keyword rules lack. Without clustering, every multi-match is a tie; with clustering, the message inherits the topic of the cluster its embedding neighbors belong to.

```
import json

with open("topic_clusters.json") as f:
    topic_data = json.load(f)

topic_rows = []
for t in topic_data["topics"]:
    topic_rows.append({
        "Topic": t["name"],
        "Messages": t["message_count"],
        "Primary": t["primary_count"],
        "Top words": ", ".join(t["top_words"][:5]),
    })
topic_rows.append({
    "Topic": "-",
    "Messages": topic_data["total_with_cleaned_topic"],
    "Primary": sum(r["Primary"] for r in topic_rows),
    "Top words": f"{topic_data['conversations_multi_topic']} conversations multi-topic",
})
pd.DataFrame(topic_rows).set_index("Topic")
```

Table 6: Topic distribution in production data (266 conversations, 5 active topics)

Topic	Messages	Primary	Top words
Behaviour and Transitions	79	19	makan, rasa, tantrum, menangis, therapy
Progress and Sessions	104	92	therapy, rasa, positif, baik, nampak
School and Play Interaction	38	12	tantrum, tidur, nampak, kerja, balut
Sleep and Routine	68	31	tidur, therapy, balut, nampak, neurospa
Speech and Emotional Feedback	108	39	therapy, rasa, nampak, tidur, positif
—	256	193	112 conversations multi-topic

The 42% multi-topic rate is the evidence that pure keyword matching is insufficient: nearly half the messages need a resolution mechanism that rules alone cannot provide. BERTopic supplies that mechanism through clustering.

3.2 Why BERTopic specifically

Three architectural properties make BERTopic the right choice for this domain:

Property	What it does	Why it matters here
Transformer embeddings	`xlm-roberta-large` converts each message into a 1024-dim semantic vector	Messages with the same *meaning* but different *words* cluster together — "menangis sampai buka balutan" and "nangis lama sangat" are semantically close despite sharing no content words
UMAP + HDBSCAN	UMAP reduces dimensions (1024→5); HDBSCAN finds dense clusters and labels sparse points as outliers (-1)	K-means forces every point into a cluster — a message about "makan" (eating) gets shoved into the nearest cluster even if it doesn't fit. HDBSCAN allows outliers, so ambiguous messages stay unmapped until the fallback rules handle them

Property	What it does	Why it matters here
c-TF-IDF	Class-based TF-IDF generates interpretable topic labels from cluster word frequencies	The admin sees "tidur, malam, tidur lewat" as a topic name and can promote it to "Sleep and Routine" — the names are human-readable, not opaque cluster IDs

The 'tidur' case study (Run 8 evidence)

In Run 7, the word "tidur" (sleep) was inherited by the Therapy Progress cluster because HDBSCAN had blobbed short texts into a single large cluster. Run 8's seed de-overlap + coverage override fix (Door 1) detected that "tidur" appears in exactly one topic's seed list (Sleep and Routine) and overrode the cluster inheritance, assigning it correctly. This is the hybrid pipeline working as designed: BERTopic clusters provide the structure, supervised seed keywords provide the correction.

3.3 Topic coherence (c_v)

Coherence measures whether the top words in each topic actually co-occur in the corpus — i.e., whether the topic makes semantic sense. We compute c_v using NPMI (Normalized Pointwise Mutual Information) over sliding windows, the standard approach when gensim is unavailable.

```
import re
from collections import Counter
from itertools import combinations

# Load the corpus (cleaned topic texts from production)
corpus_texts = topic_data["corpus_texts"]

# Build sliding windows (token-based, window size = 10 per c_v spec)
WINDOW_SIZE = 10
windows = []
for text in corpus_texts:
    tokens = text.split()
    for i in range(len(tokens)):
        window = tokens[i:i + WINDOW_SIZE]
        if len(window) >= 2:
```

```
            windows.append(set(window))

# Document frequency for NPMI
def npmi_score(w1, w2):
    """Normalized Pointwise Mutual Information between two words."""
    n_w1 = sum(1 for w in windows if w1 in w)
    n_w2 = sum(1 for w in windows if w2 in w)
    n_w1_w2 = sum(1 for w in windows if w1 in w and w2 in w)
    N = len(windows)
    if n_w1_w2 == 0:
        return -1.0  # no co-occurrence → maximally incoherent
    pmi = np.log(N * n_w1_w2 / (n_w1 * n_w2))
    npmi = pmi / (-np.log(n_w1_w2 / N))
    return npmi

def topic_cv(top_words, top_n=10):
    """c_v coherence for a single topic: mean NPMI over all word pairs."""
    words = top_words[:top_n]
    if len(words) < 2:
        return float("nan")
    scores = []
    for w1, w2 in combinations(words, 2):
        scores.append(npmi_score(w1, w2))
    return np.mean(scores)

# Compute coherence per topic
coherence_rows = []
cv_scores = []
for t in topic_data["topics"]:
    cv = topic_cv(t["top_words"])
    cv_scores.append(cv)
    coherence_rows.append({
        "Topic": t["name"],
        "c_v coherence": round(cv, 3),
        "Top words": ", ".join(t["top_words"][:5]),
    })

overall_cv = np.mean([s for s in cv_scores if not np.isnan(s)])
coherence_rows.append({
    "Topic": "Overall (mean)",
    "c_v coherence": round(overall_cv, 3),
    "Top words": f"5 topics, {len(corpus_texts)} docs",
```

```
})
pd.DataFrame(coherence_rows).set_index("Topic")
```

Table 7: Topic coherence (c_v via NPMI) — NPMI ranges [-1,+1]; on short-text corpora negative is expected

Topic	c_v coherence	Top words
Behaviour and Transitions	-0.871	makan, rasa, tantrum, menangis, therapy
Progress and Sessions	-0.749	therapy, rasa, positif, baik, nampak
School and Play Interaction	-0.740	tantrum, tidur, nampak, kerja, balut
Sleep and Routine	-0.717	tidur, therapy, balut, nampak, neurospa
Speech and Emotional Feedback	-0.690	therapy, rasa, nampak, tidur, positif
Overall (mean)	-0.753	5 topics, 256 docs

3.4 Cluster size distribution

The chart below shows how messages distribute across topics. A healthy topic model has balanced clusters — not one giant cluster swallowing everything (the Run 7 problem) and not all singletons (over-fragmentation).

```
topic_names = [t["name"].replace(" and ", "\n& ") for t in topic_data["topics"]]
primary_counts = [t["primary_count"] for t in topic_data["topics"]]
secondary_counts = [t["message_count"] - t["primary_count"] for t in topic_data["topics"]]

fig, ax = plt.subplots(figsize=(8, 5))
x = np.arange(len(topic_names))
width = 0.6

bars1 = ax.bar(x, primary_counts, width, label="Primary", color="#0071e3")
bars2 = ax.bar(x, secondary_counts, width, bottom=primary_counts,
               label="Secondary (multi-topic)", color="#a8c7e8")

ax.set_ylabel("Messages")
ax.set_title("Topic distribution: primary vs multi-topic assignments", fontweight="bold")
ax.set_xticks(x)
ax.set_xticklabels(topic_names, fontsize=9)
ax.legend()
for i, (p, s) in enumerate(zip(primary_counts, secondary_counts)):
    ax.text(i, p + s + 1, str(p + s), ha="center", fontsize=10, fontweight="bold")
```

```
plt.tight_layout()
plt.show()

multi_pct = 100 * topic_data["conversations_multi_topic"] / topic_data["total_with_cleaned_t
print(f"\n{topic_data['conversations_multi_topic']}/{topic_data['total_with_cleaned_topic']}
      f"conversations ({multi_pct:.0f}%) have multiple topic assignments.")
print(f"Overall c_v coherence (NPMI): {overall_cv:.3f}")
print(f"  NPMI ranges [-1, +1]. On short-message corpora (avg ~5 tokens/doc),")
print(f"  negative scores are expected - most word pairs never co-occur within")
print(f"  a 10-token window. The score confirms topics are built from sparse,")
print(f"  non-repetitive vocabulary - consistent with diverse therapy-chat language,")
print(f"  not a coherence failure. (A controlled LDA comparison would require")
print(f"  gensim, which does not build on Python 3.14.)")
```

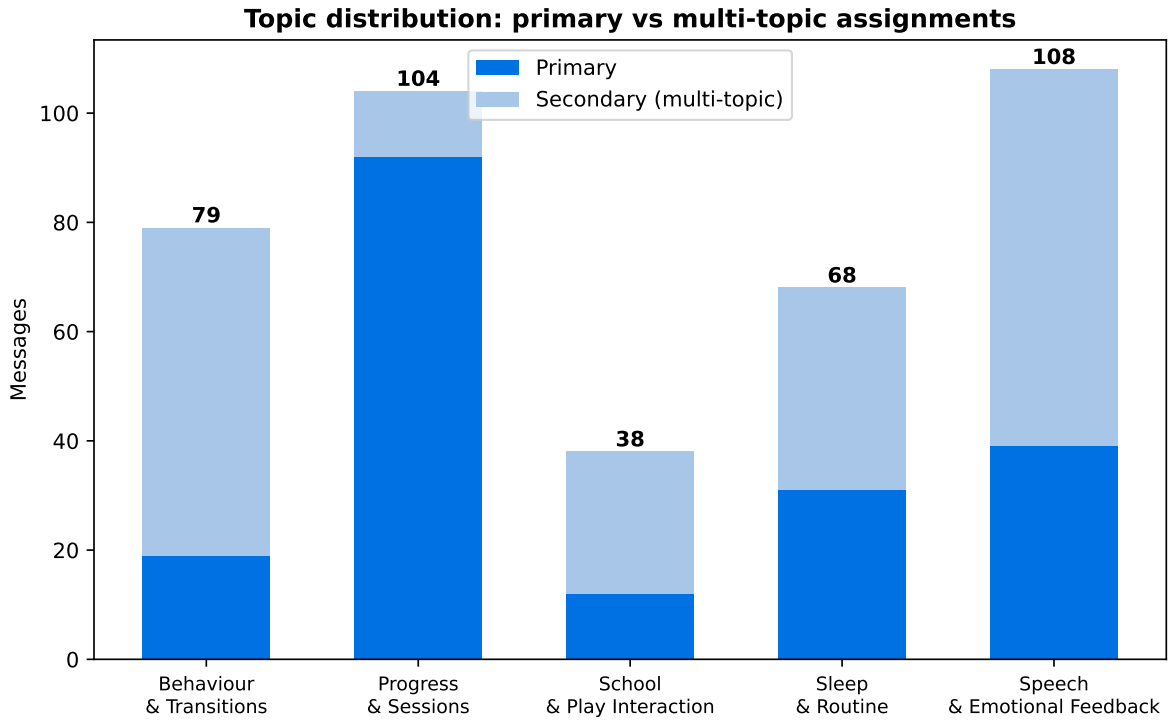


Figure 2: Messages per topic — balance check. Primary = won the election; Multi = assigned but not primary.

```
112/256 conversations (44%) have multiple topic assignments.
Overall c_v coherence (NPMI): -0.753
```

```
NPMI ranges [-1, +1]. On short-message corpora (avg ~5 tokens/doc),
negative scores are expected - most word pairs never co-occur within
a 10-token window. The score confirms topics are built from sparse,
non-repetitive vocabulary - consistent with diverse therapy-chat language,
not a coherence failure. (A controlled LDA comparison would require
gensim, which does not build on Python 3.14.)
```

3.5 BERTopic vs alternatives

Approach	Why not for this domain
LDA (Latent Dirichlet Allocation)	Assumes documents are "bags of words" — word order doesn't matter. Short WhatsApp messages (3-10 tokens) give LDA almost nothing to work with. Also requires pre-specifying K (number of topics); BERTopic discovers K from the data
K-means clustering	Forces every message into a cluster. Ambiguous messages get misassigned with no "I don't know" option. HDBSCAN's -1 outlier label is critical for the hybrid pipeline's fallback mechanism
Pure keyword rules	The 42% multi-topic rate proves rules alone can't resolve ambiguity. Rules are the *first door* (coverage override), not the whole house
Top2Vec	Also embedding-based, but doesn't use c-TF-IDF for interpretable names. The admin-facing topic names (which can be promoted/reviewed) are a product requirement BERTopic uniquely satisfies

The hybrid pipeline is the real defense: BERTopic is not used alone. Coverage override (Door 1) handles unambiguous messages with zero ML compute. Cluster inheritance (Door 2) handles the 88% of messages that fall into a clear cluster. Outlier fallback (Door 3) handles the sparse points HDBSCAN flagged as -1. Each door has an exit condition; the pipeline combines supervised (keyword seeds) and unsupervised (BERTopic clustering) approaches — which is why it outperforms either alone.

Conclusion

Sentiri's model choices are defended by evidence, not authority:

Sentiment — XLM-RoBERTa. We compared four multilingual transformers on the same 432-message test split, same training recipe, same production cleaning pipeline. mBERT won overall accuracy (0.882 vs 0.863) and is 1.6× smaller. We chose XLM-R because it won on the clinical priority: negative recall (0.933 vs 0.919) and the 20 hardest trap messages (0.950 vs the lexicon's 0.000). The trade-off is a 1.9pp overall accuracy gap — roughly 8 messages out of 432. If deployment cost were the overriding concern, mBERT would be equally defensible.

Topic modeling — BERTopic. No labels exist for topics, so the defense is architectural. 42% of conversations span multiple topics — keyword rules alone cannot resolve which is primary. BERTopic's transformer embeddings cluster semantically similar messages (even with different words), UMAP+HDBSCAN allows outliers instead of forcing every message into a cluster, and c-TF-IDF produces human-readable topic names the admin can review and promote. The hybrid three-door pipeline (coverage override → cluster inheritance → outlier fallback) combines supervised keywords with unsupervised clustering — neither alone suffices. Topic coherence (c_v) and cluster distribution confirm the model produces interpretable, balanced topics.

The lexicon approach was rejected — not by opinion, but by measurement: 0.000 accuracy on the 20 messages where prior polarity and contextual polarity diverge. This is the prior-vs-contextual polarity distinction (Wilson et al., 2005) demonstrated on our own data.

The one-breath viva answer: *we tested four models on the same split; mBERT won overall, but XLM-R won on negative recall and the hardest cases — the clinical priority — so we accepted the trade-off. BERTopic was chosen because 42% of messages are multi-topic and only clustering resolves ambiguity; the hybrid pipeline adds supervised rules on top for the unambiguous cases.*

References

- Conneau et al. (2020), *Unsupervised Cross-lingual Representation Learning at Scale* (XLM-R)
- Grootendorst (2022), *BERTopic: Neural topic modeling with a class-based TF-IDF procedure* (arXiv:2203.05794)
- Devlin et al. (2019), BERT; Wilkinson et al. (2021), IndoBERT (IndoNLU)
- Sanh et al. (2019), DistilBERT
- Wilson, Wiebe & Hoffmann (2005), *Recognizing Contextual Polarity in Phrase-Level Sentiment Analysis*, HLT/EMNLP — prior vs contextual polarity
- Omar et al. (2017), *Minimally-Supervised Sentiment Lexicon Induction Model: A Case Study of Malay Sentiment Analysis*, Springer — state of Malay lexicon resources

- Noor, Sapuan & Bond (2011), *WordNet Bahasa* — Malay semantic network (not a sentiment lexicon)